Subject: Core Java / Advanced Java GUI Programming

Topic: Introduction to JavaFX, Architecture, Components, Setup, and Programming

1. INTRODUCTION TO JAVAFX

JavaFX is a Java-based software framework used to develop graphical user interface (GUI) applications. It provides ready-made components such as buttons, labels, text fields, menus, tables, images, and other visual elements that help developers build interactive applications.

In simple language, JavaFX allows us to create software that looks like a normal computer application, with windows, buttons, forms, and menus, instead of displaying everything in a black console window.

For example, a calculator developed using basic Java may accept numbers through the console. The same calculator developed using JavaFX can display number buttons, mathematical operators, and a result field in a graphical window.

JavaFX is mainly used for desktop application development. It supports object-oriented programming, event handling, CSS styling, animations, multimedia, and the separation of user interface design from application logic.

Examples of applications that can be developed using JavaFX include:

- Student registration and management systems.
- Library management systems.
- Employee information systems.
- Billing and inventory management applications.
- Desktop calculators.
- Data visualization dashboards.
- Educational applications.
- Multimedia applications.
- Database management interfaces.

JavaFX works with Java and allows developers to use familiar concepts such as classes, objects, inheritance, methods, interfaces, and exception handling.

2. WHAT IS A GRAPHICAL USER INTERFACE (GUI)?

GUI stands for Graphical User Interface. It is a method of interacting with a computer application through visual elements such as windows, buttons, text fields, menus, checkboxes, and icons.

Consider a simple program that adds two numbers.

In a console-based application, the user enters the numbers using the keyboard, and the result is displayed as text in the console.

In a GUI-based application, the user enters the numbers into two text fields and clicks an Add button. The answer is then displayed in the application window.

The calculation may be identical in both programs, but the method of interacting with the user is different.

Important GUI elements:

- Window: The main area in which an application runs.
- Label: Displays information or instructions.
- Text field: Accepts text entered by the user.
- Button: Allows the user to perform an action.
- Menu: Provides a list of available commands.
- Table: Displays data in rows and columns.
- Dialog box: Displays a message or requests information from the user.

JavaFX provides classes for creating and managing these elements.

3. WHY DO WE NEED JAVAFX?

Java is a general-purpose programming language. It can be used to create console programs, desktop applications, web backends, and many other types of software.

However, Java's core language does not, by itself, provide every component needed to create a complete graphical interface. A GUI framework supplies the additional classes and tools required to build that interface.

JavaFX addresses this requirement by providing a structured way to create graphical applications.

The major reasons for using JavaFX are explained below.

3.1 Easy creation of graphical interfaces

JavaFX provides ready-made controls such as buttons, labels, text fields, checkboxes, and tables. Developers do not need to draw every component manually.

3.2 Interactive applications

JavaFX supports event handling. An application can respond to button clicks, text input, mouse actions, and other user interactions.

3.3 Attractive user interfaces

JavaFX supports CSS styling. Developers can customize fonts, colors, backgrounds, borders, and other visual properties.

3.4 Layout management

JavaFX provides layout containers that automatically arrange components. This makes it easier to create forms, dashboards, and data-entry screens.

3.5 Animation and multimedia

JavaFX provides facilities for animations, transitions, audio, and video playback.

3.6 Separation of design and logic

FXML allows developers to describe the user interface separately from the Java code that handles its behavior.

3.7 Integration with Java

JavaFX can use Java classes and libraries to implement calculations, business rules, file operations, and database connectivity.

4. HISTORY AND DEVELOPMENT OF JAVAFX

JavaFX was introduced by Sun Microsystems in 2008 as a technology for building rich user interfaces. Oracle later continued its development.

Earlier versions of Java included JavaFX as part of the Java platform. Starting with JDK 11, JavaFX was separated from the JDK and became available as an independent set of libraries.

OpenJFX is the open-source project that provides JavaFX. Developers can download JavaFX separately or use a build tool such as Maven or Gradle to manage the required libraries.

Today, JavaFX is used for developing Java desktop applications that require graphical controls, custom styling, interactive layouts, and other rich interface features.

5. JAVA SWING AND JAVAFX

Java Swing and JavaFX are technologies used to develop graphical applications in Java.

Java Swing is an older GUI toolkit included in the Java platform. It provides components such as JButton, JLabel, JTextField, and JTable.

JavaFX is a more modern GUI framework that provides a scene graph, CSS styling, animation support, multimedia facilities, and FXML-based interface design.

The important differences are:

Java Swing:

- Part of the Java desktop API.
- Widely used in existing Java desktop applications.
- Provides traditional GUI components.
- Uses its own layout and event-handling mechanisms.

JavaFX:

- Distributed separately from the JDK in modern Java releases.
- Provides a scene graph to organize visual elements.
- Supports CSS for styling the interface.
- Supports FXML for describing the interface.
- Provides built-in animation and multimedia APIs.

Both technologies can be useful. Swing is important when maintaining older applications, while JavaFX is a suitable choice for learning modern Java GUI development.

6. IMPORTANT FEATURES OF JAVAFX

6.1 Rich GUI controls

JavaFX provides controls such as Label, Button, TextField, PasswordField, CheckBox, RadioButton, ComboBox, ListView, and TableView.

These controls help developers create forms, menus, lists, and data-entry interfaces.

6.2 Scene graph

JavaFX organizes visual elements in a hierarchical structure called a scene graph. A window may contain a scene, which contains a layout pane, which in turn contains buttons and labels.

6.3 CSS styling

CSS stands for Cascading Style Sheets. JavaFX uses CSS rules to customize the appearance of components, including font size, text color, background color, and borders.

6.4 FXML

FXML is an XML-based format for describing JavaFX user interfaces. It helps separate interface design from application logic.

6.5 Event handling

JavaFX allows applications to respond to events such as clicking a button, selecting an item, or entering text.

6.6 Layout management

JavaFX includes layout containers such as VBox, HBox, BorderPane, GridPane, and StackPane. These containers arrange controls within the application window.

6.7 Animation and transitions

JavaFX supports animations such as fading, rotation, scaling, and movement. These can be used to create visual effects.

6.8 Multimedia support

JavaFX includes APIs for playing audio and video, subject to supported media formats and runtime requirements.

6.9 Property binding

JavaFX supports properties and binding, which can keep values synchronized between interface components. For example, one label can display a value that changes when another component's value changes.

7. UNDERSTANDING THE JAVAFX ARCHITECTURE

A JavaFX application typically contains a window, a scene, and a collection of visual elements.

The main components are Stage, Scene, Node, and Layout Pane.

7.1 Stage

Stage represents a window in a JavaFX application.

A typical application has a primary stage, which is provided to the application's start() method. An application can also create additional stages when multiple windows are required.

Examples:

- Main application window.
- Login window.
- Settings window.
- Report window.

7.2 Scene

Scene represents the content displayed inside a stage. It contains a root node, which may be a layout pane or another visual component.

For example, a scene can contain a VBox that holds a label, a text field, and a button.

7.3 Node

A Node is a basic element of the JavaFX scene graph. Buttons, labels, text fields, images, and layout panes are examples of nodes.

7.4 Layout Pane

A layout pane arranges its child components according to a particular layout strategy.

For example, VBox arranges its children vertically, while HBox arranges its children horizontally.

7.5 Scene graph structure

A basic JavaFX interface may follow this structure:

Stage | Scene | VBox | |---- Label | |---- TextField | |---- Button

In this structure, the stage represents the window, the scene contains the interface, and the VBox arranges the label, text field, and button.

This hierarchy makes it easier to organize and manage a complex interface.

8. JAVAFX APPLICATION LIFE CYCLE

A standard JavaFX application follows a defined life cycle.

8.1 main() method

The main() method is the conventional entry point of the Java program. It commonly calls launch(args) to start the JavaFX application runtime.

8.2 init() method

The init() method can be used for initialization before the user interface is started. It is optional to override this method.

8.3 start() method

The start(Stage stage) method is the main method for creating the user interface. The JavaFX runtime calls it after initialization.

Developers create controls, arrange layouts, create the scene, set the scene on the stage, and display the stage inside this method.

8.4 stop() method

The stop() method can be overridden to perform cleanup when the application is shutting down. For example, an application may close resources or stop background services.

The life cycle can be represented as:

main() → launch() → init() → start() → Application runs → stop()

The init() and stop() methods are optional to override, but start() is required when extending Application.

9. IMPORTANT JAVAFX CONTROLS

Controls are the visible components that allow users to view information or interact with an application.

Label: Displays text. Example: "Enter your name".

Button: Performs an action when activated. Example: "Submit".

TextField: Accepts a single line of text. Example: Student name or enrolment number.

PasswordField: Accepts password input and masks the displayed characters.

TextArea: Accepts multiple lines of text. Example: Address or feedback.

CheckBox: Allows an option to be selected or deselected. Example: "I agree to the terms".

RadioButton: Allows selection of an option, commonly used with a ToggleGroup when only one choice should be selected. Example: Male, Female, or Other.

ComboBox: Displays a drop-down list of options. Example: Select a department.

ListView: Displays a list of items. Example: List of courses.

TableView: Displays structured information in rows and columns. Example: Student records.

ImageView: Displays images in the user interface.

MenuBar: Provides menus such as File, Edit, and Help.

Slider: Allows users to select a value from a range.

ProgressBar: Displays the progress of an operation.

These controls can be combined to create complete graphical applications.

10. LAYOUTS IN JAVAFX

A layout determines how controls are positioned and arranged in a window. JavaFX layouts help create interfaces that remain organized when the window size changes.

10.1 VBox

VBox arranges its children vertically.

Example: A login form containing a username field, a password field, and a login button.

10.2 HBox

HBox arranges its children horizontally.

Example: A row containing Save, Update, and Cancel buttons.

10.3 BorderPane

BorderPane divides the available space into five regions: top, bottom, left, right, and center.

Example: A desktop application with a menu at the top, navigation on the left, and main content in the center.

10.4 GridPane

GridPane arranges components in rows and columns.

Example: A calculator interface with number buttons arranged in a grid.

10.5 StackPane

StackPane places its children in a stack, commonly centered by default.

Example: Displaying a message over an image.

10.6 FlowPane

FlowPane arranges components sequentially and moves them to another row or column when space is insufficient.

Example: Displaying a collection of buttons or images.

Selecting the correct layout makes the interface easier to develop and maintain.

11. EVENT HANDLING IN JAVAFX

Event handling is the process of responding to user actions.

For example, when a user clicks the Add button in a calculator, the application reads the input values, calculates the sum, and displays the result.

This is an example of event-driven programming. Instead of executing every action in a fixed sequence, the application waits for an event and then executes the relevant code.

Common events include:

- Button click.
- Mouse click.
- Keyboard input.
- Selection from a ComboBox.
- Changes to a text field.
- Window closing.

The setOnAction() method is commonly used to register an action handler for buttons.

A lambda expression can be used to write the handler concisely. For example:

```
button.setOnAction(event -> label.setText("Welcome!"));
```

When the button is activated, the label displays "Welcome!".

12. CSS STYLING IN JAVAFX

CSS is commonly used in web development to style HTML pages. JavaFX also supports CSS, but it uses JavaFX-specific property names for many controls.

For example, the following CSS rule sets the font size and text color of a label:

```
.title-label
```